

Routing3D C++ 라우팅 엔진 — 개발자 라이브러리 레퍼런스

대상: `routing3d_capi.dll`(C ABI) 및 `cpp/include/routing3d/*.hpp`(헤더 전용 C++ API)를 호출하는 개발자.

단위: 모든 좌표·치수 **mm**. 격자 6-직교(맨해튼) 이동. 좌표계 Z-업.

권위 소스: `cpp/capi/routing3d_capi.h`, `cpp/include/routing3d/`. 본 문서는 그 시그니처를 요약·해설한다.

마지막 갱신: 2026-06-12

0. 목차

1. 개요와 아키텍처
2. 빌드·링크·배포
3. 좌표·격자·단위 규약
4. 전체 라우팅 프로세스(단계별)
5. 핵심 알고리즘
6. C ABI 레퍼런스(라이브러리 표면)
7. 사용 예제
8. C++ 헤더 API(코어 직접 사용)
9. 불변식·결정성
10. 환경변수 튜닝
11. 실패 사유·진단

1. 개요와 아키텍처

Routing3D 엔진은 플랜트 배관의 **3D 직교(맨해튼) 자동 라우팅**을 수행한다.

장애물(AABB)·작업(시작/끝 PoC)을 입력받아, 충돌 없는 6-직교 경로를 산출한다.

```
호스트(C# WPF / Python / C)          ← P/Invoke·ctypes·직접 링크
|   C ABI (extern "C", cdecl, UTF-8, POD)
|   ▼
routing3d_capi.dll (R3dEngine 불투명 핸들)
|   헤더 전용 C++ 템플릿(백엔드 무관)
```

```

▼
cpp/include/routing3d/
  geometry      Cell·Vec3·AABB·6/26-이웃·맨해튼
  occupancy     DenseOccupancy / SparseOccupancy / ImplicitOccupancy
  cost          RouteParams · clearance_map · CostModel
  astar         astar · astar_weighted (상태=(셀, 진입방향))
  corridor      astar_hashed · route_corridor (계층 coarse→fine)
  multi_route   route_sequential · route_ripup · order_indices · mark_pipe
  route_task    RouteTask (작업 1 건)
  scene_io      SceneDoc · scene.txt 직렬화

```

핵심 설계 원칙:

- **헤더 전용 템플릿:** 점유 백엔드(**Occ**)에 대해 컴파일타임 다형성. A*·비용·다중배관 코드가 백엔드와 무관(불변식 O1/F2).
- **C ABI 안전:** C++ 예외는 경계를 넘지 않는다(모두 **R3dStatus** 로 보고). STL 미노출(불투명 핸들 + POD + 원시 배열). cdecl·x64·UTF-8.
- **결정성:** 동일 입력 → 동일 경로/확장수(불변식 A2/W1). tie-break = (f, 삽입순서 counter).
- **무손실 기본값:** 모든 신규 기능(가중 A*·corridor·gap·min-straight·CBS)은 **기본값=기존 동작**. 고면 골든 바이트 불변.

2. 빌드·링크·배포

DLL 빌드 (외부 의존성 0)

```

cmake -S cpp -B cpp/build -G "Visual Studio 17 2022" -A x64
cmake --build cpp/build --config Release --target routing3d_capi
# 산출물: cpp/build/Release/routing3d_capi.dll (+ routing3d_capi.lib import)

```

요구: MSVC VS2022, C++20, **/utf-8**(한글 주석), **x64 고정**(호스트 비트 일치). capi 는 코어만 링크 → OpenVDB/FCL 등 외부 의존성 없는 단일 DLL.

헤더 전용 C++ 사용

DLL 없이 엔진을 직접 쓰려면 **cpp/include** 를 include 경로에 추가하고 헤더만 포함하면 된다(라이브러리 링크 불필요).

```

#include "routing3d/scene_io.hpp"
#include "routing3d/multi_route.hpp"
using namespace routing3d;

```

검증

```
cctest --test-dir cpp/build -C Release --output-on-failure # 13/13 기대
cctest --test-dir cpp/build -C Release -R "capi|golden|ripup" # 빠른 핵심 검증
```

3. 좌표·격자·단위 규약

개념	정의
단위	모든 입력/출력 좌표·치수 = mm
격자	origin(mm) + cell_mm(셀 한 번) + shape(nx,ny,nz 셀 수)
셀↔월드	<code>to_world(cell)</code> = 셀 중심 월드좌표. <code>to_cell(world)</code> = 포함 셀 인덱스(floor)
이동	6-직교($\pm x, \pm y, \pm z$). 대각선 없음. 한 칸 = cell_mm
6-이웃 순서	<code>NEIGHBORS_6</code> = 0:+x, 1:-x, 2:+y, 3:-y, 4:+z, 5:-z (결정성. <code>goal_dir</code> 축 인덱스)
선형 인덱스	<code>lin(c) = i + nx*(j + ny*k)</code> . A*.clearance 의 상태 키. 항상 long long 으로 다룬다(A4)

격자 크기 산정: 작업·장애물의 공간 AABB 를 cell_mm 으로 나눠 shape 결정. 셀 수가 5M 을 넘으면 capi 가 자동으로 메모리 효율 백엔드(implicitOccupancy)로 전환한다(\$5.2).

4. 전체 라우팅 프로세스(단계별)

C ABI 기준 한 배치(batch)의 표준 흐름:

[1] r3d_create()	엔진 핸들 생성
[2] r3d_set_grid(g)	격자 메타(origin·cell·shape) 설정
[2] r3d_set_params(p)	비용 파라미터(회전·클리어런스·가중·회랑·랙)
[3] r3d_add_obstacle(...) × N	장애물 AABB(충돌 대상)
[3] r3d_add_passthrough(...) × M	통과 객체(가시화만, 충돌 아님)
[4] t = r3d_add_task(sx..gz, util, grp)	작업 추가 → task index

[4] r3d_set_task_diameter(t, d)	(선택) 관경 - 굵은 배관 우선 정렬
[4] r3d_set_task_goal_dir(t, axis)	(선택) 목표 진입축 제약
[5] (선택) 튜닝 setter	
r3d_set_per_task_radius(1)	관경별 마킹 반경(B1)
r3d_set_pipe_gap(60)	배관 이격(센터선 $\geq r1+r2+60$)
r3d_set_min_straight(2.0)	코너 최소반경(짧은 단관 흡수, C2)
r3d_set_cbs_depth(2)	협상 라우팅(CBS-lite, C1)
r3d_set_corridor_cells(ijk,n)	학습 회랑 시드(L2b, w_corridor>0 필요)
[6] 라우팅 (택 1)	
r3d_route_multi(priority)	순차·충돌없음(표준)
r3d_route_multi_progress(pri,cb,u)	순차 + 진행/취소 콜백
r3d_route_ripup(pri,rounds,ripup)	순차 후 rip-up 회복
r3d_route_corridor_multi(f,r,pri,pr)	계층 corridor(대형/정밀 격자)
r3d_route_task(t, &res)	단일(원본 장애물, 충돌회피 없음)
[7] r3d_get_result(t, &res)	작업별 결과 지표
[7] n = r3d_copy_path(t, NULL, 0)	경로 셀 수 조회
[7] r3d_copy_path(t, buf, n)	경로 셀(i,j,k) 복사
[7] r3d_copy_visited / r3d_copy_blocked	가시화 레이어
[8] r3d_destroy(e)	해제

핵심: `route_multi` 는 우선순위 순서로 한 배관씩 라우팅하고, 깎아낸 경로를 점유에 추가해 다음 배관이 피하게 한다(셀 공유 0 = 충돌 0, 불변식 M1). 원본 점유맵은 변경하지 않는다(내부 사본 사용, M2).

5. 핵심 알고리즘

5.1 A* (균일 / 가중 / 동적 가중)

상태 = (셀, 진입방향 `dir`). 같은 셀이라도 들어온 방향이 다르면 다른 상태 → 회전 비용을 정확히 반영. 상태 키 = `lin*7 + (dir+1)`, `dir` ∈ [-1,5].

- **균일 A*** `astar(occ, start, goal, step_cost, max_expansions, collect_visited)`: 회전 비용 없음, admissible, 최단. 골든 검증용.
- **가중 A*** `astar_weighted(occ, start, goal, params, ...)`: 비용 모델(회전 `w_turn` + 클리어런스 `w_clear` + 회랑 `w_corridor` + 랙 `w_tier`) 적용. 휴리스틱 가중 `w_heur`:
- `w_heur = 1.0` → 표준 A*(admissible, 최적, 골든 불변).
- `w_heur > 1.0` → 목표 지향(ϵ -greedy). 확장 노드 급감(어려운 우회를 상한 내 발견), 약간 비최적.

- 동적(수렴) 가중: $w_{\text{heur_near}} \in (0, w_{\text{heur}})$ 이면 목표까지 남은 거리 비율로 가중 보간:

$$w_{\text{eff}} = w_{\text{heur_near}} + (w_{\text{heur}} - w_{\text{heur_near}}) \cdot (h / h_{\text{start}})$$

먼 곳 = w_{heur} (빠름), 목표 근처 = $w_{\text{heur_near}}$ (신중, 막다른길 함정 회피). **무제한 탐색**($\text{max_expansions} \leq 0$)에서만 적용(예산-게이트 거대격자에선 자동 비활성).

tie-break = (f 오름차순, 삽입 counter 오름차순) + 고정 이웃 순서 → **완전 결정적**.

진행/취소: $\text{on_progress}(\text{expanded}, \text{progress01}) \rightarrow \text{bool}$. $\text{progress01} = 1 - h_{\text{min}}/h_{\text{start}}$. 반환 true = 즉시 취소(실패 결과 반환). 약 5 만 확장마다 호출(C ABI 콜백이 여기에 연결).

목표 진입축(goal_dir): 목표 도달을 $\text{cur.cell} == \text{goal} \ \&\& \ (\text{goal_dir} < 0 \ || \ \text{cur.dir} == \text{goal_dir})$ 로 제한. 덩트 스텝 리드인 축을 주면 일직선 진입. 막히면 무제약 1 회 폴백.

5.2 점유 백엔드 3 종 (백엔드 무관 계약)

모든 백엔드는 동일 질의

계약($\text{in_bounds} \cdot \text{is_blocked} \cdot \text{to_world} \cdot \text{to_cell} \cdot \text{block_cell} \cdot \text{add_box} \cdot \text{lin} \cdot \text{unlin} \cdot \text{copy} \cdot \text{size}$)을 만족 → A*비용 코드 공유. **동일 입력 → 동일 결과**(O1).

백엔드	메모리	용도	키 타입
DenseOccupancy	O(전체 셀), 1B/셀	소격자-골든(바이트 일치)	int lin
SparseOccupancy	O(점유 셀)	초대형 격자 corridor	64 비트 패킹
ImplicitOccupancy	O(장애물 + 깔린 셀)	정밀-거대 격자(복셀화 폐기, 장애물 AABB 를 유니폼그리드 색인)	long long lin

capi 는 격자 > **5M 셀**이면 자동으로 Dense→Implicit 전환(메모리 폭발 방지). 5M 이하는 Dense(골든 바이트 불변).

5.3 비용 모델 / 클리어런스

RouteParams (§8.2)의 가산 페널티만 사용 → **admissibility 보존**(휴리스틱이 과소평가 유지). 구성:

- `w_turn` × 방향 전환 횟수 — 꺾임 억제.
- `w_clear` × (`clearance_radius` - 거리) — 장애물에서 떨어지게(여유 확보). `clearance_map` = 다중소스 BFS 거리변환(연결성 6/26). 거대격자에선 **온디맨드 클리어런스 질의**(전역 거리변환 폐기).
- `w_corridor` × 회랑 밖 셀 — 회랑(학습/자기번들) 곁으로 유도. 0 = 비활성.
- `w_tier`(z 셀→mm) / `rack_levels` — 선호 단(랙 높이)로 유도.

5.4 다중배관: 순차 / rip-up / CBS-lite

- **순차**(`route_sequential` / `route_multi`): 우선순위 순서로 한 배관씩. 성공 경로를 `mark_pipe`(경로 셀 + 반경 팽창)로 점유 → 다음 배관 충돌 0. 원본 불변.
- **rip-up & reroute**(`route_ripup`): 순차 후 막힌 배관 `f` 의 '장애물-only 이상경로'를 가로막는 placed 배관(blocker, $\leq \text{max_ripup}$)을 뜯어 재배치. `f` 성공 + 모든 blocker 재배치 성공일 때만 채택. **무손실(성공 단조 +1)·결정적·유한 종료**.
- **CBS-lite**(`r3d_set_cbs_depth`): rip-up 의 재귀 확장. blocker 가 재배치 못 하면 그 blocker 의 blocker 까지 bounded depth(≤ 3) 양보. conflict-based search 경량판. 무손실·결정적.

5.5 계층 corridor (대형 장면)

`route_corridor` / `astar_hashed`: coarse 격자에서 가이드 경로 → fine 튜브($\pm \text{radius}$)로 하드 제한해 A* 탐색량 축소. **해시 기반**(셀 수 배열 미할당) → 8,000m³ 등 초대형 격자도 동작. 저예산 직접 A* 먼저 → 초과하는 어려운 배관만 escalate.

5.6 배관 물리(관경)

- **per-task 반경**(`r3d_set_per_task_radius`): 각 배관의 마킹 반경 = `clamp(ceil(d/cell)-1, 0, 8)`. 가는 배관이 굵은 배관 반경으로 과패킹되던 문제 해소.
- **배관 이격**(`r3d_set_pipe_gap`): 두 배관 센터선 거리 $\geq r_1 + r_2 + \text{gap}$. 깔린 배관을 쌍 반경 `ceil((r_a + r_b + gap)/cell)` 로 막아 표면 사이 최소 gap 확보(규격 60mm).
- **코너 최소반경**(`r3d_set_min_straight`): 엘보 사이 직선 < (mult × 관경)인 짧은 단관을 경로 단계에서 충돌검사 하에 코너로 흡수(꺾임 비증가일 때만, 양 끝점 고정). **비교란 최종 패스**.

5.7 우선순위 정렬 (`order_indices`)

priority	규칙
longest	맨해튼 거리 긴 것 먼저(기본)
shortest	짧은 것 먼저
diameter	굵은 배관 먼저, 동률은 거리 긴 것. 관경 0 이면 longest 와 동일
utility	(유틸 라벨 ↑, 굵은 배관 먼저, 거리 ↓). 유틸 묶음 유지 + 묶음 안 굵은 배관 먼저
original	입력 순서

안정 정렬(stable_sort) = Python sorted 결정성과 1:1.

6. C ABI 레퍼런스(라이브러리 표면)

모든 함수는 extern "C", cdecl, x64. 문자열은 UTF-8. 반환 R3dStatus 또는 정수(인덱스/셀 수, 음수=오류).

6.0 상태 코드 · 버전 · 메모리

```
typedef enum {
    R3D_OK = 0, R3D_ERR_ARG = 1, R3D_ERR_PARSE = 2,
    R3D_ERR_RUNTIME = 3, R3D_ERR_RANGE = 4
} R3dStatus;

const char* r3d_version(void);          // 정적 문자열(해제 불필요)
void r3d_free_string(char* s);         // 콜리 할당 문자열 해제(Level 1·dump 출력)
```

6.1 생명주기

함수	설명
R3dEngine* r3d_create(void)	엔진 핸들 생성. 실패 시 NULL
void r3d_destroy(R3dEngine* e)	핸들·내부 리소스 해제

함수	설명
<code>R3dStatus r3d_load_scene_text(R3dEngine* e, const char* scene_text)</code>	scene.txt(UTF-8) 로 격자·장애물·작업 일괄 적재

6.2 씬 구성

```
typedef struct { double cell_mm; double ox, oy, oz; int32_t nx, ny, nz; } R3dGrid;

typedef struct {
    double cell_mm, w_turn, w_clear;
    double w_corridor; // 회랑 밖 셀 가산 mm. 0=비활성
    double w_heur; // 휴리스틱 가중. 1=표준(골든 불변), >1=목표지향
    double w_heur_near; // 동적 가중 목표근처 값. (0,w_heur)=수렴, 0=정적
    int32_t clearance_radius, clearance_connectivity; // 6 또는 26
    int32_t corridor_radius; // 회랑 성장 반경(셀). 기본 1
    int32_t rack_level_count; // rack_levels 사용 개수(0~8)
    int32_t rack_levels[8]; // 선호 단(z 셀 인덱스). 회랑 면제
} R3dParams;
```

함수	설명
<code>R3dStatus r3d_set_grid(e, const R3dGrid* g)</code>	격자 메타 설정. 라우팅 전 필수
<code>R3dStatus r3d_set_params(e, const R3dParams* p)</code>	비용 파라미터 설정
<code>R3dStatus r3d_add_obstacle(e, minx,miny,minz, maxx,maxy,maxz)</code>	장애물 AABB(충돌 대상)
<code>R3dStatus r3d_add_passthrough(e, minx,miny,minz, maxx,maxy,maxz)</code>	통과 객체(가시화만, 충돌 아님)

6.3 작업

함수	설명
<code>int32_t r3d_add_task(e, sx,sy,sz, gx,gy,gz, utility, utility_group)</code>	작업 추가 → task index(≥0) 반환, 실패 음수. util/grp = 색 분류(NULL 허용)
<code>R3dStatus r3d_set_task_endpoints(e, task, sx,sy,sz, gx,gy,gz)</code>	종단점 갱신(인터랙티브 편집)

함수	설명
<code>R3dStatus r3d_set_task_diameter(e, task, diameter_mm)</code>	관경 — <code>diameter/utility</code> 정렬에서 굵은 배관 우선. 0=무시
<code>R3dStatus r3d_set_task_goal_dir(e, task, axis)</code>	목표 진입축($0.5=\pm x, \pm y, \pm z$). -1=무제약(기본·골든 불변)

6.4 라우팅

함수	설명
<code>R3dStatus r3d_route_multi(e, priority)</code>	전체 순차·충돌없음. priority NULL→"longest"
<code>R3dStatus r3d_route_multi_progress(e, priority, R3dProgressFn cb, void* user)</code>	순차 + 배관별 진행/취소 콜백(\$6.7)
<code>R3dStatus r3d_route_ripup(e, priority, max_rounds, max_ripup)</code>	순차 후 rip-up 회복. 무손실·결정적
<code>R3dStatus r3d_route_corridor(e, factor, radius)</code>	계층 corridor(대형, 작업별 독립·충돌회피 없음)
<code>R3dStatus r3d_route_corridor_multi(e, factor, radius, priority, pipe_radius)</code>	순차 계층 corridor + 충돌회피
<code>R3dStatus r3d_route_task(e, task, R3dResult* out)</code>	단일 작업(원본 장애물, 다른 배관 무시)

6.5 튜닝 setter (모두 기본값=기존 동작·골든 불변)

함수	기본	설명
<code>r3d_set_pipe_radius(e, radius_cells)</code>	0	깔린 배관 $\pm radius$ 팽창(글로벌). env <code>R3D_PIPE_RADIUS</code>

함수	기본	설명
<code>r3d_set_per_task_radius(e, enabled)</code>	0	관경별 반경 자동 산출(B1). env <code>R3D_PER_TASK_RADIUS</code>
<code>r3d_set_pipe_gap(e, gap_mm)</code>	0	센터선 $\geq r1+r2+gap$ (규격 60). env <code>R3D_PIPE_GAP</code>
<code>r3d_set_min_straight(e, mult)</code>	0	짧은 단관(< mult×관경) 코너 흡수(C2, 권장 2.0). env <code>R3D_MIN_STRAIGHT</code>
<code>r3d_set_cbs_depth(e, depth)</code>	0	협상 라우팅 재귀 깊이 0,3. env <code>R3D_CBS</code>
<code>r3d_set_corridor_cells(e, ijk, n)</code>	없음	학습 회랑 시드(L2b). <code>w_corridor>0</code> 필요. <code>n≤0</code> =비움
<code>r3d_set_collect_visited(e, enabled)</code>	1	방문맵 수집 on/off(대형 메모리 절약 시 0)

6.6 결과·가시화 조회

```
typedef struct {
    int32_t success;           // 1/0
    double length_mm;         // 기하 길이
    double cost_mm;           // 페널티 포함 총비용
    int32_t turns;
    int64_t expanded_nodes;
    double elapsed_ms;
    int32_t path_len;         // 경로 셀 수(copy_path 버퍼 산출)
    int32_t visited_len;      // 방문 셀 수(비활성 시 0)
    int32_t fail_reason;      // 실패 사유(§11). success=0 일 때만 의미
} R3dResult;
```

함수	설명
<code>R3dStatus r3d_get_result(e, task, R3dResult* out)</code>	작업별 결과 지표
<code>int32_t r3d_copy_path(e, task, int32_t* buf, int32_t buf_cells)</code>	경로 셀(i,j,k 연속)을 buf 에 복사. 반환=복사 셀 수. <code>buf=NULL,0</code> → 총 셀 수만
<code>int32_t r3d_copy_visited(e, task, buf, buf_cells)</code>	방문(확장) 셀 복사(방문맵)

함수	설명
<code>int32_t r3d_copy_blocked(e, buf, buf_cells)</code>	점유(블록) 셀 복사(점유맵). buf=NULL,0 → 총 셀 수
<code>int32_t r3d_copy_passthrough(e, buf, buf_cells)</code>	통과 객체 점유 셀 복사
<code>R3dStatus r3d_dump_scene_text(e, char** out_text)</code>	현재 상태 → scene.txt(UTF-8). out_text 는 <code>r3d_free_string</code> 해제

2 단계 버퍼 규약: 셀 배열은 콜러 할당. 먼저 길이(`R3dResult.path_len` 또는 `copy_*` NULL 호출)로 크기를 알아내고, `int32_t[3*n]` 할당 후 다시 호출한다.

6.7 진행·취소 콜백

```
typedef int32_t(__cdecl* R3dProgressFn)(
    void* user, int32_t phase, int32_t order_index, int32_t task_index,
    int32_t success, double length_mm, int32_t turns, int64_t expanded_nodes,
    double elapsed_ms, int32_t done, int32_t total, double progress01,
    const int32_t* path_ijk, int32_t path_len);
```

인자	의미
<code>phase</code>	0=탐색 진행(progress01 %), 1=배관 완료(지표+경로)
<code>order_index</code>	처리 순서(priority 정렬 기준, 0 부터)
<code>task_index</code>	원본 작업 인덱스(<code>get_result</code> 와 동일 매핑)
<code>success/length_mm/turns/...</code>	phase==1 결과 지표(phase==0 은 expanded 만 유효)
<code>done/total</code>	진행률(완료 배관 수 / 전체)
<code>progress01</code>	phase==0 탐색 진행율(0~1). phase==1 은 1.0
<code>path_ijk/path_len</code>	phase==1 성공 시 경로 셀. 콜백 동안만 유효 → 즉시 복사

인자	의미
반환	0=계속, 0 아님=취소(abort). 현재 배관 즉시 중단, 완료분 보존, 나머지 미라우팅, R3D_OK 정상 반환

협력적(cooperative) 취소: 탐색 중(약 5 만 확장마다)·배관 완료마다 반환값 검사.

7. 사용 예제

7.1 C — 최소 라우팅

```
#include "routing3d_capi.h"
#include <stdlib.h>
#include <stdio.h>

int main(void) {
    R3dEngine* e = r3d_create();

    // 1) 격자: 원점(0,0,0), 셀 50mm, 200×200×40 셀
    R3dGrid g = { 50.0, 0,0,0, 200,200,40 };
    r3d_set_grid(e, &g);

    // 2) 비용 파라미터(회전 억제 + 약간의 여유)
    R3dParams p = {0};
    p.cell_mm = 50.0; p.w_turn = 500.0; p.w_clear = 10.0;
    p.w_heur = 1.0; // 표준 A*(최적)
    p.clearance_radius = 2; p.clearance_connectivity = 6;
    p.corridor_radius = 1;
    r3d_set_params(e, &p);

    // 3) 장애물 1 개(AABB, mm)
    r3d_add_obstacle(e, 2000,2000,0, 3000,3000,2000);

    // 4) 작업 2 건
    int t0 = r3d_add_task(e, 100,100,500, 9000,9000,500, "NW", "Water");
    int t1 = r3d_add_task(e, 100,9000,500, 9000,100,500, "RW", "Water");
    r3d_set_task_diameter(e, t0, 150.0);
    r3d_set_task_diameter(e, t1, 100.0);

    // 5) 물리 옵션: 관경별 반경 + 60mm 이격 + 굵은 배관 우선
    r3d_set_per_task_radius(e, 1);
    r3d_set_pipe_gap(e, 60.0);

    // 6) 라우팅(굵은 배관 먼저)
    if (r3d_route_multi(e, "diameter") != R3D_OK) { r3d_destroy(e); return 1; }

    // 7) 결과 + 경로 복사(2 단계 버퍼)
    R3dResult r;
    r3d_get_result(e, t0, &r);
}
```

```

    printf("t0 success=%d len=%.0fmm turns=%d cells=%d\n",
           r.success, r.length_mm, r.turns, r.path_len);

    int n = r.path_len;
    int32_t* buf = (int32_t*)malloc(sizeof(int32_t) * 3 * n);
    int got = r3d_copy_path(e, t0, buf, n);
    for (int i = 0; i < got; ++i)
        printf("  cell %d: (%d,%d,%d)\n", i, buf[3*i], buf[3*i+1], buf[3*i+2]);
    free(buf);

    r3d_destroy(e);
    return 0;
}

```

7.2 C# — P/Invoke 발췌

```

[StructLayout(LayoutKind.Sequential)]
struct R3dGrid { public double cell_mm, ox, oy, oz; public int nx, ny, nz; }

[StructLayout(LayoutKind.Sequential)]
struct R3dResult {
    public int success; public double length_mm, cost_mm; public int turns;
    public long expanded_nodes; public double elapsed_ms;
    public int path_len, visited_len, fail_reason;
}

const string DLL = "routing3d_capi";
[DllImport(DLL, CallingConvention = CallingConvention.Cdecl)] static extern IntPtr
r3d_create();
[DllImport(DLL, CallingConvention = CallingConvention.Cdecl)] static extern void
r3d_destroy(IntPtr e);
[DllImport(DLL, CallingConvention = CallingConvention.Cdecl)] static extern int
r3d_set_grid(IntPtr e, ref R3dGrid g);
[DllImport(DLL, CallingConvention = CallingConvention.Cdecl)]
static extern int r3d_add_task(IntPtr e, double sx,double sy,double sz, double
gx,double gy,double gz,
    [MarshalAs(UnmanagedType.LPUTF8Str)] string? util,
    [MarshalAs(UnmanagedType.LPUTF8Str)] string? grp);
[DllImport(DLL, CallingConvention = CallingConvention.Cdecl)]
static extern int r3d_route_multi(IntPtr e, [MarshalAs(UnmanagedType.LPUTF8Str)]
string? priority);
[DllImport(DLL, CallingConvention = CallingConvention.Cdecl)]
static extern int r3d_get_result(IntPtr e, int task, out R3dResult res);
[DllImport(DLL, CallingConvention = CallingConvention.Cdecl)]
static extern int r3d_copy_path(IntPtr e, int task, int[]? buf, int bufCells);

// 사용
var e = r3d_create();
var g = new R3dGrid { cell_mm = 50, nx = 200, ny = 200, nz = 40 };
r3d_set_grid(e, ref g);
int t = r3d_add_task(e, 100,100,500, 9000,9000,500, "NW", "Water");
r3d_route_multi(e, "diameter");
r3d_get_result(e, t, out var res);
var buf = new int[3 * res.path_len];
int got = r3d_copy_path(e, t, buf, res.path_len);    // (buf[3i],buf[3i+1],buf[3i+2]) =
i 번째 셀
r3d_destroy(e);

```

실제 뷰어 래퍼는 `csharp/Routing3D.Viewer/Interop/{Native,Engine}.cs` 참조(전체 ABI + 안전 래퍼).

7.3 문자열 ABI (Level 1) — 한 줄 라우팅

scene.txt 를 그대로 넣고 결과 scene.txt 를 받는 가장 단순한 진입점:

```
char* out = NULL;
// mode = "multi"(순차·충돌없음) | "single". priority NULL → "longest".
R3dStatus st = r3d_route_scene_text(scene_text_utf8, "multi", "longest", &out);
if (st == R3D_OK) { /* out = 결과 scene.txt(경로 레이어 포함) */ }
r3d_free_string(out);
```

`diameter` 정렬·관경별 반경·이격 등 세부 튜닝은 Level 2 핸들 ABI(`r3d_create` + setter)를 쓴다. Level 1 문자열 ABI 는 `mode/priority` 만 받는 간편 진입점이다.

8. C++ 헤더 API(코어 직접 사용)

DLL 없이 헤더만으로 엔진을 쓸 때의 핵심 타입·함수. 모두 `namespace routing3d`.

8.1 RouteTask (`route_task.hpp`)

```
struct RouteTask {
    Vec3 start_mm, end_mm; // 월드좌표 mm
    std::optional<std::string> utility, utility_group, start_name, end_name,
end_instance_guid;
    double diameter_mm = 0.0; // 정렬 키(굵은 배관 우선).
scene.txt 미직렬화
    int goal_dir = -1; // 목표 진입축(0..5). -1=무제약
    std::string utility_label() const; // "[그룹] 유틸"
};
```

8.2 RouteParams (`cost.hpp`)

```
struct RouteParams {
    double cell_mm = 50.0;
    double w_turn = 500.0; // 회전 1 회 가산 mm
    double w_clear = 10.0; // 클리어런스 부족 가산
    double w_heur = 1.0; // 휴리스틱 가중(1=최적·골든 불변, >1=목표지향)
    double w_heur_near = 0.0; // 동적 가중 목표근처 값(0,w_heur)=수렴, 0=정적
    int clearance_radius = 2;
    int clearance_connectivity = 6; // 6 또는 26
    std::map<int,double> w_tier; // z 셀 → 가산 mm
    double w_corridor = 0.0; // 회랑 밖 셀 가산(>0=번들링, 0=비활성)
    int corridor_radius = 1;
    std::vector<int> rack_levels; // 선호 단(z 셀). w_corridor 면제
};
```

8.3 A* (astar.hpp)

```
template <class Occ>
AStarResult astar(const Occ& occ, Cell start, Cell goal,
                  double step_cost = -1.0, long long max_expansions = -1,
                  bool collect_visited = false);

template <class Occ, class InCorridor = AllowAll>
AStarResult astar_weighted(const Occ& occ, Cell start, Cell goal, const RouteParams&
params,
                           long long max_expansions = -1, bool collect_visited =
false,
                           const std::unordered_set<long long>* corridor = nullptr,
                           const std::function<bool(long long,double)>* on_progress =
nullptr,
                           long long progress_every = 0, InCorridor in_corridor = {},
                           int goal_dir = -1);

struct AStarResult {
    bool success; std::vector<Cell> path;
    double length_mm; int turns; long long expanded_nodes;
    double cost_mm, elapsed_ms; RouteFail fail;
    std::vector<Cell> visited; // collect_visited=true 일 때만
};
```

8.4 다중배관 (multi_route.hpp)

```
template <class Occ>
MultiRouteResult<Occ> route_sequential(
    const Occ& occ, const std::vector<RouteTask>& tasks, const RouteParams& params,
    const std::string& priority = "longest", int pipe_radius = 0, int snap_to_free =
2,
    long long max_expansions = -1, bool collect_visited = false, int corridor_radius =
1);

template <class Occ>
MultiRouteResult<Occ> route_ripup(
    const Occ& occ, const std::vector<RouteTask>& tasks, const RouteParams& params,
    const std::string& priority = "longest", int pipe_radius = 0, int snap_to_free =
2,
    long long max_expansions = -1, int max_rounds = 10, int max_ripup = 4,
    bool collect_visited = false);

// 보조
std::vector<int> order_indices(const Occ&, const std::vector<RouteTask>&, const
std::string& priority);
std::vector<RouteTask> order_tasks (const Occ&, const std::vector<RouteTask>&, const
std::string& priority);
Cell snap_to_free_cell(const Occ&, Cell cell, int radius);
void mark_pipe (Occ&, const std::vector<Cell>& path, int radius);

struct MultiRouteResult<Occ> {
    std::vector<PipeResult> pipes; // 라우팅 순서대로
    Occ occupancy; // 최종 점유맵
    std::string priority;
    int success_count() const; int fail_count() const;
    double total_length_mm() const; double success_rate() const;
};
```

8.5 scene_io (scene_io.hpp)

```
struct SceneDoc {
    double cell_mm; Vec3 origin; Cell shape; RouteParams params;
    std::vector<Obstacle> obstacles, passthrough;
    std::vector<RouteTask> tasks;
    std::vector<std::optional<SceneResult>> results;
};

std::string      dumps_scene(const SceneDoc&);           // → scene.txt
void             write_scene(const std::string& path, const SceneDoc&);
SceneDoc         loads_scene(const std::string& text);  // scene.txt →
SceneDoc         read_scene (const std::string& path);
DenseOccupancy  occupancy_from_doc(const SceneDoc&);   // 점유맵 복원
std::string      format_repr_double(double);           // Python repr(float) 동일 표기
```

8.6 헤더 전용 — 직접 사용 예제

```
#include "routing3d/scene_io.hpp"
#include "routing3d/multi_route.hpp"
using namespace routing3d;

SceneDoc doc = read_scene("scene.txt");
DenseOccupancy occ = occupancy_from_doc(doc);

RouteParams p = doc.params;
p.w_turn = 500.0; p.w_clear = 10.0;

auto out = route_sequential(occ, doc.tasks, p, /*priority*/"diameter",
                             /*pipe_radius*/0, /*snap_to_free*/2);

printf("success %d/%zu total %.0fmm\n",
       out.success_count(), out.pipes.size(), out.total_length_mm());

for (const PipeResult& pr : out.pipes)
    if (pr.result.success)
        printf(" order %d len %.0f turns %d\n",
               pr.order_index, pr.result.length_mm, pr.result.turns);
```

9. 불변식·결정성

ID	불변식
A2/W1	A* 결정성: (f, 삽입 counter) tie-break + 고정 이웃 순서 → 동일 입력 = 동일 경로/확장수
M1	다중배관 성공 경로 셀 공유 0(충돌 0)

ID	불변식
M2	원본 점유맵 불변(<code>copy()</code> 사본으로 작업)
O1	백엔드(Dense/Sparse/Implicit) 무관 동일 결과
F2	scene.txt 바이트 무손실 왕복
F3	<code>optional<string></code> 로 <code>None(\N)</code> vs 빈문자열("") 구분
F4	<code>format_repr_double</code> = Python <code>repr(float)</code>
골든 불변	모든 신규 기능 기본값=기존 동작 → <code>w_heur=1</code> · <code>gap=0</code> · <code>min_straight=0</code> · <code>cbs=0</code> · <code>corridor</code> 없음이면 바이트 동일. ctest 13/13

엔진 수정 후 반드시 `ctest --test-dir cpp/build -C Release(13/13)`. capi 수정 시 뷰어 bin DLL 복사 필수(cpp/build 만 빌드하면 stale).

10. 환경변수 튜닝

대부분의 setter 는 env 로도 재정의 가능(헤드리스 A/B 비교용). C# 진단 `Routing3D.Viewer.exe --dbroute <proj> <cell> <util> <out>` 에서 활용.

env	대응 setter / 의미
<code>R3D_PIPE_RADIUS</code>	<code>r3d_set_pipe_radius</code> 글로벌 팽창 반경
<code>R3D_PER_TASK_RADIUS</code>	<code>r3d_set_per_task_radius</code> (on/off)
<code>R3D_PIPE_GAP</code>	<code>r3d_set_pipe_gap</code> 이격 mm
<code>R3D_MIN_STRAIGHT</code>	<code>r3d_set_min_straight</code> 배수
<code>R3D_CBS</code>	<code>r3d_set_cbs_depth</code> 깊이[0,3]

env	대응 setter / 의미
R3D_MAX_EXP	A* 확장 상한(거대격자, 기본 48M)
R3D_WHEUR / R3D_WHEUR_NEAR	w_heur / w_heur_near
R3D_WCORR / R3D_CORRRAD	w_corridor / corridor_radius
R3D_RIPUP	인라인 rip-up on/off
R3D_PRIORITY	우선순위 규칙(diameter/longest/...)
R3D_GOALDIR	목표 진입축 제약 on/off(실험)

11. 실패 사유·진단

R3dResult.fail_reason(= RouteFail, success=0 일 때만):

값	enum	의미	대응
0	None	성공/미시도	—
1	StartBlocked	시작 셀이 막힘(장애물/격자 밖)	시작 PoC 가 장애물에 매물 → 스냅 반경 ↑ / PoC 좌표 점검
2	GoalBlocked	목표 셀이 막힘	종단 PoC 매물 → 동일
3	CorridorMiss	시작/목표가 하드 corridor(튜브) 밖	corridor 시드 범위 확장 또는 무제약 풀백
4	ExpansionLimit	확장 상한 도달(거대격자 어려움/사실상 막힘)	R3D_MAX_EXP ↑ / 가중 A*(w_heur>1) / hier corridor

값	enum	의미	대응
5	GoalDirBlocked	목표엔 닿았으나 요구 진입축으로 못 들어감	<code>goal_dir</code> 무제약 풀백(자동)
6	NoPath	탐색 고갈(연결 경로 없음=국소 차단)	rip-up/CBS 회복 대상(혼잡/막힘)

진단 절차:

1. `fail_reason` 으로 1 차 분류(매물 1/2 ↔ 혼잡 6 ↔ 상한 4).
2. `r3d_copy_blocked` 로 점유맵 가시화 → PoC 주변 막힘 확인.
3. 상한(4)·혼잡(6)은 `route_ripup` / `r3d_set_cbs_depth` 로 회복 시도.

부록 A. 관련 문서

문서	내용
<code>docs/spec/algorithm_spec.md</code> 외 4 종	Phase 2 동결 명세(불변식 포함)
<code>docs/routing3d_cpp_engine_spec.{docx,pdf}</code>	C++ 엔진 상세 개발문서(프로세스·알고리즘·함수, 13 장)
<code>docs/routing3d_engine_improvement_plan.md</code>	엔진 고도화 Phase A~D 계획·실측
<code>docs/csharp_helix_interop_design.md</code>	C ABI·뷰어 인터럽 설계
<code>cpp/capi/routing3d_capi.h</code>	C ABI 권위 헤더 (본 문서의 1 차 출처)
<code>cpp/include/routing3d/*.hpp</code>	코어 C++ 헤더(템플릿 구현)

부록 B. 빠른 체크리스트

- [] `r3d_set_grid` → `r3d_set_params` 순서로 호출(라우팅 전).

- [] 장애물/작업 추가 후 라우팅. task index 는 `add_task` 반환값 사용.
- [] 경로 조회는 **2 단계 버퍼**(길이 먼저 → 할당 → 복사).
- [] 문자열은 UTF-8. 콜리 할당 문자열은 `r3d_free_string`.
- [] 콜백 반환 0=계속/0 아님=취소. `path_ijk` 는 즉시 복사.
- [] 신규 기능은 기본 OFF=골든 불변. 켜 때만 setter 호출.
- [] capi 수정 후 ctest 13/13 + 뷰어 bin DLL 갱신.